

ФГБОУ ВО «АДЫГЕЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ НАУК

Кафедра прикладной математики, информационных технологий и
информационной безопасности

Отчет

о выполнении лабораторной работы №1 по предмету

«Теория вычислительных процессов и структур»

Коновалов Ярослав Александрович

студент 4 курса очного отделения

направление 02.03.03 «Математическое обеспечение и администрирование
информационных систем»

направленность: «Технологии программирования»

Майкоп 2024

ОГЛАВЛЕНИЕ

Задание по лабораторной работе	3
Схема организации хеш-таблицы	4
Описание алгоритма поиска в хеш-таблице	6
Текст программы	7
Выводы по проделанной работе	9

Задание по лабораторной работе

Вариант №3. Тип хеш-функции (таблицы): Сумма кодов первой и второй букв;

Способ разрешения коллизий: упорядоченный список с логарифмическим поиском.

Требуется разработать программу, реализующую комбинированный способ организации таблицы идентификаторов. Для организации таблицы используется простейшая хеш-функция, указанная в варианте задания, а при возникновении коллизий используется дополнительный метод размещения идентификаторов в памяти. Если в качестве этого метода используется дерево или список, то они должны быть связаны с элементом главной хэш-таблицы.

В каждом варианте требуется, чтобы программа сообщала среднее число коллизий и среднее количество сравнений, выполненных для поиска идентификатора.

Схема организации хеш-таблицы

В своём варианте реализации программы я организовал хеш-таблицу в виде массива строк размерностью 149 элементов. Данный размер был рассчитан следующим образом: я предположил, что область значений выбранной хеш-функции будет в следующем диапазоне: $(\text{ord}('0') + \text{ord}('0')) .. (\text{ord}('z') + \text{ord}('z'))$.

Максимальное значение суммы ASCII кодов пары символов будет составлять $\text{ord}('z') + \text{ord}('z') = 122 + 122 = 244$.

Минимальное значение суммы ASCII кодов пары символов будет составлять $\text{ord}('0') + \text{ord}('0') = 48 + 48 = 96$.

Таким образом, область значений суммы ASCII кодов пары символов из диапазона '0'..'z' будет варьироваться от 96 до 244. Отсюда следует, что всего будет $244 - 96 + 1 = 149$ различных значений.

Массив размерностью 244 делать нецелесообразно в связи с тем, что минимальная сумма ASCII кодов будет равна 96. Значит, элементы в диапазоне индексов $[0..96]$ будут постоянно пустыми, что будет являться невыгодным использованием оперативной памяти компьютера.

В связи с этим было принято следующее дополнение хеш-функции: от суммы ASCII кодов будем отнимать минимальную сумму ASCII кодов равную 96. В таком случае, размер массива можно сократить до 149, и теперь есть шанс того, что все ячейки массива будут заполнены какими-либо данными, что говорит о том, что пустующих ячеек не останется, а значит оперативная память не будет использована просто так.

Используемая хеш-функция в данном варианте задания является суммой кодов первой и второй букв идентификатора. Но, в связи с вышеперечисленными обстоятельствами, было принято решение дополнить её разницей минимальной возможной суммы кодов букв, что не должно повлиять на конечный результат (оставь функцию как есть, пришлось бы увеличить размер массива и оставить

много пустых ячеек, но дальнейшая работа программы была бы абсолютно идентичной).

Используемый метод разрешения коллизий в данном варианте является использование упорядоченного списка с логарифмическим поиском. Данный способ был реализован мной следующим способом:

Если при очередном заполнении хеш-таблицы была обнаружена занятая ячейка по вычисленному адресу, то создавался отдельный список с тем же самым адресом, куда записывался очередной идентификатор, после чего список сортировался в алфавитном порядке для дальнейшего логарифмического поиска.

Описание алгоритма поиска в хеш-таблице

Поиск осуществляется следующим образом:

1. Находим хеш-значение (адрес) искомой строки;
2. Проверяем строку в хеш-таблице по найденному адресу. Если строка совпадает – строка найдена, иначе переходим к Шагу 3;
3. Проверяем список по найденному адресу начиная с его середины. Если строка совпадает – строка найдена, иначе проверяем блок элементов, пронумерованных от 0 до $(n + 1) / 2$, или блок элементов от $(n + 1) / 2 + 1$ до $n - 1$ в зависимости от того, меньше искомая строка или больше того, с которой её сравнивали. Затем повторяем процесс над блоком меньшего размера.

После выполнения алгоритма, программа выдает результат в зависимости от успешности нахождения идентификатора, а также выдает информацию о количестве проведенных сравнений.

Текст программы

Программа была реализована на языке программирования C++/CLI с использованием Windows Forms.

Начальные данные:

```
string table[149] = {};           - таблица хеш-значений
list<string> list1[149] = {};     - список
int freeCount = 0;               - счетчик свободных ячеек в таблице
int totalCompareCount = 0;       - общий счетчик сравнений
int searchCount = 0;             - текущий счетчик сравнений
```

Функция получения хеш-значения

```
int get_hash(string A) {
    if (A.length() != 1) {
        return ((int)A[0] + (int)A[1]) - (int)'0' - (int)'0';
    }
    else {
        return ((int)A[0] + (int)A[0]) - (int)'0' - (int)'0';
    }
}
```

Функция заполнения хеш-таблицы

```
void fill_table(string A, int& count, int& free) {
    int h = get_hash(A);

    if (table[h] == "") {
        table[h] = A;
        free--;
    }
    else {
        list1[h].push_back(A);
        list1[h].sort();
        count++;
    }
}
```

Функция очистки хеш-таблицы

```
void clear_table() {
    list1->clear();
    for (int i = 0; i <= (sizeof(table) / sizeof(table[0])); i++) {
        table[i] = "";
    }
}
```

Функция нахождения идентификатора

```
bool find_text(string A, int& count) {
    int h = get_hash(A);
    count++;

    if (table[h] == A) {
        return true;
    }
    else if (!list1[h].empty()) {
        list<string> list2 = list<string>(list1[h].begin(),
list1[h].end());
        int low = 0;
        int high = list2.size()-1;

        while (low <= high) {
            count++;
            int mid = low + (high - low) / 2;

            auto i = list2.begin();
            advance(i, mid);

            if (*i == A) {
                return true;
            }
            else if (*i < A) {
                low = mid + 1;
            }
            else {
                high = mid - 1;
            }
        }
    }

    return false;
}
```


Выводы по проделанной работе

В ходе данной работы мной были изучены основные методы организации таблиц идентификаторов, было получено представление о преимуществах и недостатках, присущих различным методам организации таблиц идентификаторов.

Полученные результаты позволяют сделать вывод о том, что комбинированный метод организации таблицы идентификаторов, использующий простейшую хеш-функцию и упорядоченный список с логарифмическим поиском, обеспечивает эффективное управление памятью и быстрый доступ к данным при поиске идентификаторов.